

Pull Request Intelligence Platform — Design Document

1. Understanding & Assumptions

Problem: Build a platform that integrates with source control (GitHub/GitLab), collects PR data, computes quality metrics, and surfaces insights at the PR level and the engineer level.

Assumptions made (since the brief is intentionally open-ended):

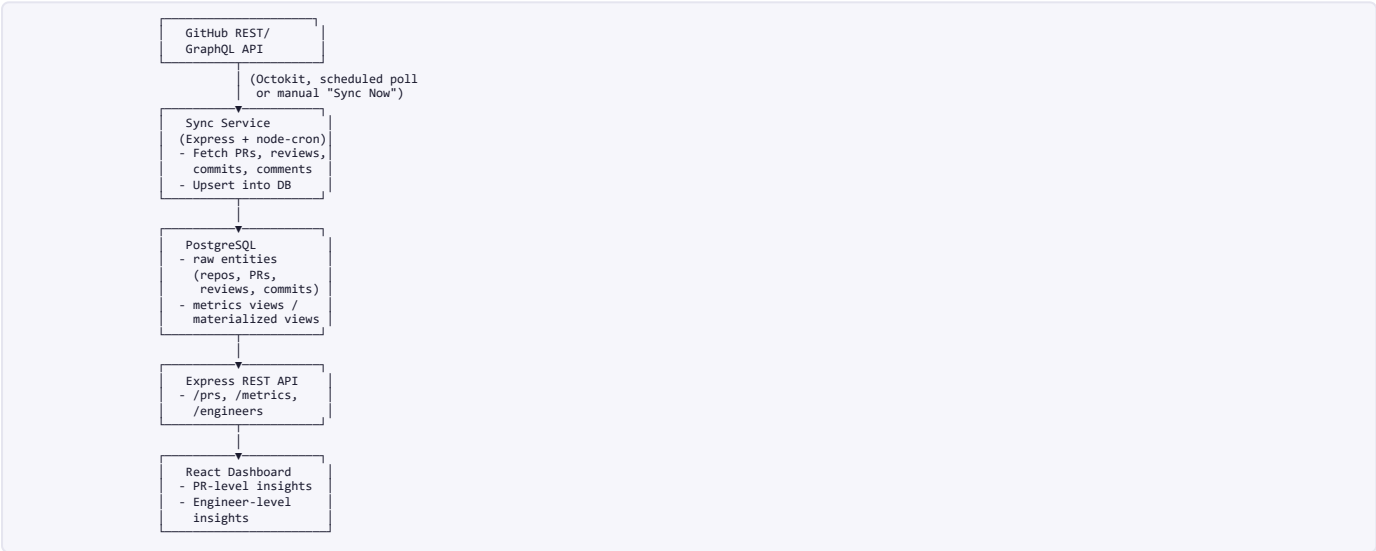
- **Source control provider:** GitHub is the primary target (REST/GraphQL via Octokit). GitLab follows the same pattern via an adapter — not built in MVP, but the data model and ingestion layer are designed to be provider-agnostic.
- **Scope of "engineer":** An engineer is identified by their GitHub username/email and may map to multiple repos within an organization.
- **Data freshness:** Near-real-time is not a hard requirement for MVP. A periodic sync (e.g., every 15–30 min) or on-demand "Sync Now" is acceptable. Webhooks are a post-MVP enhancement.
- **Auth:** A GitHub Personal Access Token (or OAuth App for multi-user) is sufficient for MVP; fine-grained per-user permissions are out of scope.
- **"Quality metrics":** Interpreted as a mix of **process metrics** (cycle time, review responsiveness) and **change metrics** (PR size, rework, comment density) — not static code analysis (linting/coverage), which would require a separate CI integration and is flagged as a future extension.
- **Audience:** Engineering managers/leads viewing trends across repos and individuals — not meant to be a performance-ranking tool, framed as a process-health dashboard.

Open questions for stakeholders (would clarify before building beyond MVP):

1. Single org/repo set, or multi-tenant across many orgs?
2. Should metrics be configurable (e.g., custom SLA thresholds for "stale PR")?
3. Is GitLab support a near-term requirement or nice-to-have?
4. Any compliance/privacy constraints on storing engineer-level performance data?

2. Proposed Solution & Architecture

High-level architecture



Core data model (simplified)

- `repositories` (id, name, provider, external_id)
- `pull_requests` (id, repo_id, number, title, author, state, created_at, merged_at, closed_at, additions, deletions, changed_files)
- `reviews` (id, pr_id, reviewer, state, submitted_at)
- `review_comments` (id, pr_id, author, created_at)
- `commits` (id, pr_id, author, committed_at)
- `engineers` (id, username, display_name) — derived/normalized from authors across PRs/reviews

Metrics layer

Computed in the **application layer as pure, isolated functions** (one `*.metrics.ts` per domain module), which keeps the highest-risk logic trivial to unit-test exhaustively without a database. As data volume grows, hot aggregations can move into SQL/materialized views refreshed after each sync (see tradeoffs):

PR-level:

- Time to first review, time to merge, review count
- PR size (lines changed, files changed) and a size bucket (XS/S/M/L/XL)
- Comment count and density (across conversation, inline, and review-summary feedback)

Engineer-level:

- Total / merged PRs, avg & median cycle time (open → merge)
- Avg PR size
- Review load given vs. received

Repository-level: PR state breakdown, size distribution, and **weekly throughput trend** (PRs merged per week).

(Planned extension: rework rate — commits pushed after first review.)

AI Insight Layer (Future Enhancement)

On top of the deterministic metrics, a hybrid AI layer can generate higher-level insights without sending raw repo data to an LLM on every request:

- **PR summarization & risk scoring:** an AI worker consumes structured metrics + PR metadata (size, files touched, review cycles, missing tests) to produce a plain-language summary and a risk score (e.g., "High risk — auth module, 45 files, no test changes, 3 review rounds").

- **Team-level trend insights:** periodic (e.g., weekly) AI-generated summaries such as "review turnaround up 18% this sprint" or "review load concentrated on 2 engineers."
- **Why hybrid:** deterministic SQL metrics stay cheap and instant; the LLM only processes pre-aggregated summaries on a schedule or on-demand, keeping cost predictable and outputs explainable (every AI claim traces back to a metric).
- **Architecture impact:** introduce a queue (BullMQ/Redis) with a separate **AI Worker** alongside the **Metrics Worker**, so AI calls (latency/cost) never block ingestion or API responses.

3. MVP Scope

In scope:

- Connect to one GitHub org/repo set via PAT
- Manual "Sync Now" trigger + optional scheduled cron sync (ingest PRs, reviews, comments, commits)
- Postgres schema + metrics computed in the service layer
- REST API: list PRs with computed metrics, per-engineer profile, repo-level summary
- React dashboard: PR table (with links to GitHub), engineer profiles with charts, repo summaries with size/state distribution charts

Explicitly out of scope for MVP (called out as future work):

- GitLab/Bitbucket adapters
- Webhooks / real-time updates
- Multi-tenant org management & user auth/roles
- Static code analysis (lint, test coverage, security scans)
- Configurable SLAs/alerting

4. Implementation Approach

Stack: Node.js + Express, PostgreSQL + Prisma (ORM + migrations), React (Vite) + Recharts, Octokit for GitHub API.

Phased delivery:

1. **Schema & ingestion:** Prisma schema, sync service pulling PRs/reviews/comments/commits for configured repos, idempotent upserts (by external GitHub IDs).
2. **Metrics:** pure functions per domain (unit-tested with mocked repositories); exposed via the service layer.
3. **API:** REST endpoints — `GET /api/pull-requests`, `GET /api/pull-requests/:id`, `GET /api/engineers`, `GET /api/engineers/:username`, `GET /api/repositories`, `GET /api/repositories/:id/summary`, `POST /api/github/sync`.
4. **Dashboard:** React app — PR table (with GitHub links), engineer profiles with charts, repo summaries with size/state distribution charts.

Why this order: Data model and ingestion are the foundation everything else depends on; getting raw data correct early de-risks the metrics layer, which can then iterate quickly once data is in place.

A working reference implementation of this MVP has been built (Node/Express + Prisma + React), with unit tests covering the metric-calculation logic and validated against a live GitHub repository.

5. Key Tradeoffs & Scalability Considerations

Decision	Tradeoff	Rationale
Polling vs. webhooks	Polling is simpler, no public endpoint needed, slightly stale data	Acceptable for MVP; webhooks + queue (BullMQ/Redis) are the natural next step for real-time, high-volume orgs
App-layer metric functions vs. SQL views	Computing in code recomputes on read and pulls rows into the app; simpler and unit-testable, but can get slow at scale	Chose pure functions for testability/DRY at MVP volumes; move hot aggregations to materialized views (refreshed post-sync) or scheduled aggregation jobs as data grows
Single-tenant (PAT) vs. OAuth multi-tenant	PAT is simplest but not scalable to many users/orgs	Flagged as a clear, isolated upgrade path (swap auth layer, add org_id scoping)
Express monolith vs. separate sync worker	Monolith is simpler to deploy/run for MVP	At scale, split sync into a separate worker process/service so ingestion load doesn't impact API latency
GitHub API rate limits	Large orgs/repos can hit rate limits during full sync	Use incremental sync (only fetch PRs updated since last sync via <code>updated_at</code> filter) and GraphQL to batch requests
Plaintext PAT vs. encrypted secrets	Plaintext is faster to set up for MVP	PAT should be encrypted at rest (or stored in a secrets manager) before any non-local deployment
Engineer-level metrics framing	Risk of misuse as a performance-ranking tool	Present as process-health indicators (team/repo trends) rather than individual rankings; access-controlled in production

Scaling path summary: monolith → separate sync worker + queue (with AI worker for insights) → webhook-driven ingestion → materialized/aggregated metrics tables → multi-tenant auth & provider adapters (GitLab, etc.).